

KODE LENGKAP KLASIFIKASI

ECG Image Classification Pipeline

CNN Baseline vs Hybrid CNN + Transformer

Generated: 2026-07-26 11:37:24

DAFTAR ISI:

- Part 1: Data Loading & Preprocessing
- Part 2: Data Augmentation (Class-Specific)
- Part 3: Model Architectures (CNN Baseline & Hybrid CNN+Transformer)
- Part 4: Training Loop
- Part 5: 5-Fold Cross-Validation
- Part 6: Evaluation & Visualization
- Main Execution

Fitur Utama:

- Custom PyTorch Dataset dengan preprocessing otomatis (grayscale, CLAHE, resize 224×224)
- Class-specific augmentation (Abnormal 1.5× lebih agresif)
- CNN Baseline: ResNet18 transfer learning (layer4 trainable)
- Hybrid CNN+Transformer: ResNet18 + Transformer Encoder (8 heads, 2 layers)
- 5-Fold Stratified Cross-Validation dengan class-weighted loss
- Comprehensive metrics: Accuracy, Precision, Recall, F1, AUC-ROC

```
# -*- coding: utf-8 -*-
"""
EKG Image Classification: CNN Baseline vs Hybrid CNN+Transformer
Complete Pipeline: Preprocessing → Augmentation → Modeling → Training → Evaluation
"""

import os
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
from pathlib import Path
from collections import defaultdict

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, models
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, confusion_matrix,
                             roc_curve, auc, calibration_curve, brier_score_loss)
```

```
# =====
```

```
# PART 1: DATA LOADING & PREPROCESSING
```

```
# =====
class ECGImageDataset(Dataset):
    """Custom PyTorch Dataset for ECG images with preprocessing."""
    def __init__(self, image_paths, labels, transform=None):
        self.image_paths = image_paths
        self.labels = labels
        self.transform = transform
    def __len__(self):
        return len(self.image_paths)
    def __getitem__(self, idx):
        # Read image
        img_path = self.image_paths[idx]
        img = cv2.imread(img_path)
        label = self.labels[idx]
        # Preprocessing
        img = self._preprocess(img)
        # Transform (augmentation)
        if self.transform:
            img = self.transform(img)
        return torch.from_numpy(img).float(), torch.tensor(label, dtype=torch.long)
    @staticmethod
    def _preprocess(img):
        """Preprocessing: grayscale → CLAHE → resize → normalize."""
        if img is None:
            raise ValueError("Failed to read image")
        # Convert BGR to RGB then grayscale
        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        img_gray = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY)
        # CLAHE enhancement
        clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
        img_enhanced = clahe.apply(img_gray)
        # Resize to 224x224
        img_resized = cv2.resize(img_enhanced, (224, 224), interpolation=cv2.INTER_AREA)
        # Normalize to [0, 1]
        img_norm = img_resized.astype(np.float32) / 255.0
        # Convert to 3-channel (grayscale → RGB by duplication)
        img_3ch = np.stack([img_norm, img_norm, img_norm], axis=0)
        return img_3ch
def load_dataset(root_dir):
    """Load ECG images from directory structure."""
    image_paths = []
    labels = []
    label_map = {"Normal": 0, "Abnormal": 1}
    for label_name, label_id in label_map.items():
        class_dir = Path(root_dir) / label_name
        if class_dir.exists():
            for img_path in class_dir.glob("*.jpg"):
                image_paths.append(str(img_path))
                labels.append(label_id)
            for img_path in class_dir.glob("*.jpeg"):
                image_paths.append(str(img_path))
                labels.append(label_id)
    return np.array(image_paths), np.array(labels)
```

```
# =====
```

```
# PART 2: DATA AUGMENTATION
```

```
# =====

class ECGAugmentation:
    """Class-specific augmentation strategy."""
    # Base augmentation parameters (for Normal class)
    BASE_PARAMS = {
        'rotation_deg': 5,
        'shift_pct': 0.05,
        'zoom_pct': 0.05,
        'brightness_range': (0.9, 1.1),
        'noise_sigma': 0.02,
        'blur_sigma_range': (0.5, 1.5),
        'apply_prob': 0.7,
    }
    # Multiplier for Abnormal class (1.5x more aggressive)
    ABNORMAL_MULTIPLIER = 1.5
    @classmethod
    def get_transforms(cls, label, train=True):
        """Return albumentations transforms based on class and phase."""
        try:
            import albumentations as A
        except ImportError:
            print("Warning: albumentations not installed, using basic transforms")
            return transforms.Compose([
                transforms.ToTensor(),
                transforms.Normalize(mean=[0.485], std=[0.229])
            ])
        if not train:
            # Test: no augmentation
            return A.Compose([
                A.Normalize(mean=[0.485], std=[0.229]),
            ], is_check_shapes=False)
        # Training: augmentation
        mult = cls.ABNORMAL_MULTIPLIER if label == 1 else 1.0
        p = cls.BASE_PARAMS['apply_prob']
        return A.Compose([
            A.Rotate(limit=int(cls.BASE_PARAMS['rotation_deg'] * mult),
                    p=p, border_mode=cv2.BORDER_REFLECT),
            A.ShiftScaleRotate(shift_limit=cls.BASE_PARAMS['shift_pct'] * mult,
                              scale_limit=cls.BASE_PARAMS['zoom_pct'] * mult,
                              p=p),
            A.RandomBrightnessContrast(
                brightness_limit=(cls.BASE_PARAMS['brightness_range'][0]-1,
                                cls.BASE_PARAMS['brightness_range'][1]-1) * mult,
                p=p),
            A.GaussNoise(p=p, scale=cls.BASE_PARAMS['noise_sigma'] * mult * 255),
            A.GaussianBlur(sigma_limit=cls.BASE_PARAMS['blur_sigma_range'], p=p),
            A.Normalize(mean=[0.485], std=[0.229]),
        ], is_check_shapes=False)
```

=====

PART 3: MODEL ARCHITECTURES

```
# =====

class CNNBaseline(nn.Module):
    """CNN Baseline: ResNet18 with transfer learning."""
    def __init__(self, num_classes=2, pretrained=True):
        super(CNNBaseline, self).__init__()
        self.backbone = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1 if pretrained else None)
        # Freeze most layers
        for param in self.backbone.conv1.parameters():
            param.requires_grad = False
        for param in self.backbone.bn1.parameters():
            param.requires_grad = False
        for param in self.backbone.layer1.parameters():
            param.requires_grad = False
        for param in self.backbone.layer2.parameters():
            param.requires_grad = False
        for param in self.backbone.layer3.parameters():
            param.requires_grad = False
        # Replace classifier
        in_features = self.backbone.fc.in_features
        self.backbone.fc = nn.Linear(in_features, num_classes)
    def forward(self, x):
        return self.backbone(x)

class HybridCNNTransformer(nn.Module):
    """Hybrid Architecture: ResNet18 backbone + Transformer Encoder."""
    def __init__(self, num_classes=2, pretrained=True):
        super(HybridCNNTransformer, self).__init__()
        # CNN Backbone (without avgpool and fc)
        backbone = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1 if pretrained else None)
        self.features = nn.Sequential(*list(backbone.children())[:-2])
        # Freeze backbone layers except layer4
        for param in self.features[:-1].parameters():
            param.requires_grad = False
        # Learnable CLS token and positional embeddings
        self.cls_token = nn.Parameter(torch.randn(1, 1, 512))
        self.pos_embed = nn.Parameter(torch.randn(1, 50, 512)) # 49 patches + 1 CLS
        # Transformer encoder
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=512, nhead=8, dim_feedforward=1024,
            dropout=0.1, batch_first=True
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=2)
        # Classification head
        self.classifier = nn.Linear(512, num_classes)
    def forward(self, x):
        # CNN backbone
        feat_map = self.features(x) # (B, 512, 7, 7)
        # Flatten spatial dimensions
        B, C, H, W = feat_map.shape
        tokens = feat_map.view(B, C, H*W).permute(0, 2, 1) # (B, 49, 512)
        # Add CLS token
        cls_tokens = self.cls_token.expand(B, -1, -1) # (B, 1, 512)
        tokens = torch.cat([cls_tokens, tokens], dim=1) # (B, 50, 512)
        # Add positional embeddings
        tokens = tokens + self.pos_embed
        # Transformer
        encoded = self.transformer(tokens) # (B, 50, 512)
        # Use CLS token for classification
```



```
cls_out = encoded[:, 0, :] # (B, 512)
logits = self.classifier(cls_out) # (B, 2)
return logits
```

```
# =====
```

```
# PART 4: TRAINING LOOP
```

```
# =====
```

```
def train_epoch(model, train_loader, criterion, optimizer, device):
```

```
    """Train for one epoch."""
```

```
    model.train()
```

```
    total_loss = 0.0
```

```
    for images, labels in train_loader:
```

```
        images, labels = images.to(device), labels.to(device)
```

```
        optimizer.zero_grad()
```

```
        logits = model(images)
```

```
        loss = criterion(logits, labels)
```

```
        loss.backward()
```

```
        optimizer.step()
```

```
        total_loss += loss.item()
```

```
    return total_loss / len(train_loader)
```

```
def evaluate(model, data_loader, device):
```

```
    """Evaluate model on dataset."""
```

```
    model.eval()
```

```
    y_true, y_pred, y_prob = [], [], []
```

```
    with torch.no_grad():
```

```
        for images, labels in data_loader:
```

```
            images = images.to(device)
```

```
            logits = model(images)
```

```
            probs = torch.softmax(logits, dim=1)
```

```
            preds = torch.argmax(logits, dim=1)
```

```
            y_true.extend(labels.cpu().numpy())
```

```
            y_pred.extend(preds.cpu().numpy())
```

```
            y_prob.extend(probs[:, 1].cpu().numpy())
```

```
    return np.array(y_true), np.array(y_pred), np.array(y_prob)
```

=====

PART 5: 5-FOLD CROSS-VALIDATION

```
# =====
def train_with_5fold_cv(image_paths, labels, model_class, model_name,
                        num_epochs=12, batch_size=16, lr=1e-4):
    """Train and evaluate using 5-fold stratified cross-validation."""
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    metrics_list = []
    all_y_true = []
    all_y_prob = []
    for fold, (train_idx, test_idx) in enumerate(skf.split(image_paths, labels), 1):
        print(f"\n{' '*60}")
        print(f"{model_name} - Fold {fold}")
        print(f"{' '*60}")
        # Split data
        train_paths = image_paths[train_idx]
        train_labels = labels[train_idx]
        test_paths = image_paths[test_idx]
        test_labels = labels[test_idx]
        # Compute class weights
        unique, counts = np.unique(train_labels, return_counts=True)
        class_weights = torch.from_numpy(1.0 / counts).float().to(device)
        # Create datasets
        train_dataset = ECGImageDataset(train_paths, train_labels)
        test_dataset = ECGImageDataset(test_paths, test_labels)
        train_loader = DataLoader(train_dataset, batch_size=batch_size,
                                  shuffle=True, num_workers=0)
        test_loader = DataLoader(test_dataset, batch_size=batch_size,
                                  shuffle=False, num_workers=0)
        # Initialize model
        model = model_class(num_classes=2).to(device)
        optimizer = optim.Adam(filter(lambda p: p.requires_grad, model.parameters()), lr=lr)
        criterion = nn.CrossEntropyLoss(weight=class_weights)
        # Training
        best_f1 = 0
        for epoch in range(1, num_epochs + 1):
            train_loss = train_epoch(model, train_loader, criterion, optimizer, device)
            if epoch % 3 == 0 or epoch == num_epochs:
                y_true, y_pred, y_prob = evaluate(model, test_loader, device)
                f1 = f1_score(y_true, y_pred, average='binary')
                print(f"Epoch {epoch:2d} | Loss: {train_loss:.4f} | F1: {f1:.4f}")
                if f1 > best_f1:
                    best_f1 = f1
        # Final evaluation
        y_true, y_pred, y_prob = evaluate(model, test_loader, device)
        # Compute metrics
        acc = accuracy_score(y_true, y_pred)
        prec = precision_score(y_true, y_pred, average='binary')
        rec = recall_score(y_true, y_pred, average='binary')
        f1 = f1_score(y_true, y_pred, average='binary')
        auc = roc_auc_score(y_true, y_prob)
        metrics_list.append({
            'fold': fold,
            'accuracy': acc,
            'precision': prec,
            'recall': rec,
            'f1': f1,
            'auc': auc,

```

```
    })
    all_y_true.extend(y_true)
    all_y_prob.extend(y_prob)
    print(f"Accuracy:  {acc:.4f}")
    print(f"Precision: {prec:.4f}")
    print(f"Recall:    {rec:.4f}")
    print(f"F1-Score:  {f1:.4f}")
    print(f"AUC-ROC:   {auc:.4f}")
return pd.DataFrame(metrics_list), np.array(all_y_true), np.array(all_y_prob)
```

=====

PART 6: EVALUATION & VISUALIZATION

```
# =====
def compute_metrics_summary(cnn_metrics_df, hybrid_metrics_df):
    """Compute aggregated metrics across folds."""
    summary = {
        'Model': ['CNN Baseline', 'Hybrid CNN+Transformer'],
        'Accuracy (mean)': [
            f"{cnn_metrics_df['accuracy'].mean():.4f} ± {cnn_metrics_df['accuracy'].std():.4f}",
            f"{hybrid_metrics_df['accuracy'].mean():.4f} ± {hybrid_metrics_df['accuracy'].std():.4f}",
        ],
        'Precision (mean)': [
            f"{cnn_metrics_df['precision'].mean():.4f} ± {cnn_metrics_df['precision'].std():.4f}",
            f"{hybrid_metrics_df['precision'].mean():.4f} ± {hybrid_metrics_df['precision'].std():.4f}",
        ],
        'Recall (mean)': [
            f"{cnn_metrics_df['recall'].mean():.4f} ± {cnn_metrics_df['recall'].std():.4f}",
            f"{hybrid_metrics_df['recall'].mean():.4f} ± {hybrid_metrics_df['recall'].std():.4f}",
        ],
        'F1-Score (mean)': [
            f"{cnn_metrics_df['f1'].mean():.4f} ± {cnn_metrics_df['f1'].std():.4f}",
            f"{hybrid_metrics_df['f1'].mean():.4f} ± {hybrid_metrics_df['f1'].std():.4f}",
        ],
        'AUC-ROC (mean)': [
            f"{cnn_metrics_df['auc'].mean():.4f} ± {cnn_metrics_df['auc'].std():.4f}",
            f"{hybrid_metrics_df['auc'].mean():.4f} ± {hybrid_metrics_df['auc'].std():.4f}",
        ],
    }
    return pd.DataFrame(summary)

def plot_roc_curves(cnn_all_true, cnn_all_prob, hybrid_all_true, hybrid_all_prob):
    """Plot ROC curves for both models."""
    fpr_cnn, tpr_cnn, _ = roc_curve(cnn_all_true, cnn_all_prob)
    fpr_hybrid, tpr_hybrid, _ = roc_curve(hybrid_all_true, hybrid_all_prob)
    auc_cnn = auc(fpr_cnn, tpr_cnn)
    auc_hybrid = auc(fpr_hybrid, tpr_hybrid)
    plt.figure(figsize=(8, 7))
    plt.plot(fpr_cnn, tpr_cnn, color="#339af0", linewidth=2, label=f"CNN Baseline (AUC={auc_cnn:.3f})")
    plt.plot(fpr_hybrid, tpr_hybrid, color="#e8590c", linewidth=2, label=f"Hybrid CNN+Transformer (AUC={auc_hybrid:.3f})")
    plt.plot([0, 1], [0, 1], linestyle="--", color="gray", linewidth=1.5, label="Random Classifier")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title("ROC Curve - Aggregated 5-Fold Predictions")
    plt.legend(loc="lower right")
    plt.grid(alpha=0.3)
    plt.tight_layout()
    return plt.gcf()
```



```
# =====
```

```
# MAIN EXECUTION
```

```

# =====
if __name__ == "__main__":
    # Configuration
    DATA_ROOT = r"D:\PROGRAM\ekg_cnn_transformers\EKG_image"
    RESULTS_DIR = r"D:\PROGRAM\ekg_cnn_transformers\results"
    os.makedirs(RESULTS_DIR, exist_ok=True)

    print("="*70)
    print("ECG IMAGE CLASSIFICATION: CNN BASELINE vs HYBRID CNN+TRANSFORMER")
    print("="*70)
    print(f"Start Time: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
    print()

    # Load dataset
    print("Loading dataset...")
    image_paths, labels = load_dataset(DATA_ROOT)
    print(f"Total images: {len(image_paths)}")
    print(f"Class distribution: {np.bincount(labels)}")
    print()

    # Train CNN Baseline
    print("Training CNN Baseline with 5-Fold CV...")
    cnn_metrics_df, cnn_all_true, cnn_all_prob = train_with_5fold_cv(
        image_paths, labels, CNNBaseline, "CNN Baseline"
    )

    # Train Hybrid CNN+Transformer
    print("\nTraining Hybrid CNN+Transformer with 5-Fold CV...")
    hybrid_metrics_df, hybrid_all_true, hybrid_all_prob = train_with_5fold_cv(
        image_paths, labels, HybridCNNTransformer, "Hybrid CNN+Transformer"
    )

    # Summary
    print("\n" + "="*70)
    print("PERFORMANCE SUMMARY (5-Fold Cross-Validation)")
    print("="*70)
    summary_df = compute_metrics_summary(cnn_metrics_df, hybrid_metrics_df)
    print(summary_df.to_string(index=False))

    # Save results
    cnn_metrics_df.to_csv(os.path.join(RESULTS_DIR, "cnn_metrics.csv"), index=False)
    hybrid_metrics_df.to_csv(os.path.join(RESULTS_DIR, "hybrid_metrics.csv"), index=False)
    summary_df.to_csv(os.path.join(RESULTS_DIR, "summary_metrics.csv"), index=False)
    print(f"\nEnd Time: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
    print(f"Results saved to: {RESULTS_DIR}")

```